Compile-time transition safety versus exhaustive testing in composable finite-state machines: an experimental comparison in Haskell, C and C++

Peterson Carara Junior[a,*]

[a] *Faculdade de Tecnologia da Baixada Santista "Rubens Lara" (Fatec BS), [street and number to be added], Santos, SP, Brazil*

Abstract

Typed state-machine libraries promise that illegal transitions are rejected at compile time, while exhaustive testing over a finite domain is a common alternative. What does compile-time checking add? We implement one four-state machine in five ways (C with `enum` and `switch`; C++17 with `enum class`; C++17 with a type-level topology; Haskell with algebraic data types; Haskell with the `crem` library) and compare each with a transition table that records which rules come from the requirements and which are conventions. We inject defects from six classes and observe compiler diagnostics at three warning levels, a static analyser, a type checker and an exhaustive test of the 16 state–event pairs. A rejection matrix over all 16 vertex pairs is identical in `crem` and in C++17 with a type-level topology: the eight pairs outside the topology are rejected, each with the expected cause. Rejecting a forbidden transition at compile time is therefore not specific to Haskell. Omitted cases are rejected by default only by C++17 `std::visit`; elsewhere they stop the

*Corresponding author.
Email address: `[e-mail to be added]` (Peterson Carara Junior)

build only when warnings are errors, and a catch-all clause hides them from default diagnostics. No compiler or analyser detected a wrong destination on an allowed edge; only the test did. Compiler output for an omitted case ranges from 4 to 236 lines. Compile-time checking advances detection and removes edge-specific tests, but does not make behavioural correctness equivalent to type safety. The study has one machine, one compiler per language and author-written code.
Keywords: finite-state machines, Haskell, type-level programming, static verification, mutation analysis, C++, exhaustive testing

---

1. Introduction

A finite-state machine is the standard model for reactive behaviour, and programming languages differ in how much of its specification they can check before the program runs. In C and in C++ with plain enumerations, states and events are integers in disguise, and a `switch` statement is the usual encoding; exhaustiveness is, at best, a compiler warning. In a pure functional language, states and events are algebraic data types and the compiler checks that the transition function covers every pair [1]. Libraries such as `crem` [2] go further and lift the set of *allowed transitions* (the topology) to the type level, so that an action returning a destination not permitted from its origin fails to type-check.

The usual argument for such a guarantee is that it moves the detection of a defect from testing to compilation. The argument is rarely tested against its natural alternative. In a finite domain, such as the 16 state–event pairs of a small machine, an exhaustive test is cheap to write, and other constructions,

even in C++, may reject the same defects at compile time. This paper asks what compile-time checking adds, measured on a concrete machine.

1.1. Research questions

RQ1 Which classes of defect does each mechanism (compiler warnings at several levels, a static analyser, a type checker, an exhaustive test) detect, in each of five implementations?

RQ2 Is the compile-time rejection of a forbidden transition specific to Haskell and `crem`?

RQ3 What does an exhaustive test add to compile-time checking, and what can it not see?

RQ4 What does the diagnostic cost, measured as the size of the compiler output?

1.2. Contributions

1. A controlled comparison of five implementations of the same machine against one transition table, with six defect classes and four mechanisms, reproducible with a single script (Sect. 3).
2. A matrix of all 16 ordered vertex pairs, with the cause of each rejection checked, for `crem` and for a C++17 construction with a type-level topology: the outcomes are identical (Sect. 4.1).
3. The observation that, except where the encoding makes an omitted case a type error (C++17 with `std::visit`), the gap between "warns" and "rejects" for omitted cases is a matter of configuration, not of language,

and that a catch-all clause hides omissions from the default diagnostics of all five stacks (Sect. 4.2).

4. The observation that no compiler or analyser in the study detects a wrong destination on an allowed edge, so the guarantee of a type-level topology is per edge and not per event (Sect. 4.2).
5. A measurement of the size of the diagnostics, which differs by two orders of magnitude between constructions (Sect. 4.4).

1.3. Scope

The subject is a four-state machine used as an illustration; there are no safety requirements behind it, and nothing is claimed about real avionics. The paper proposes no new type system and no formal semantics. The behaviour of this machine when composed with others and fed back is the subject of a companion manuscript in preparation; the results reported here do not depend on it.

2. Background

2.1. Typed topologies in crem

A `crem` machine is a Mealy machine: it emits an output and updates its state on each input. Its vertices are promoted to the type level with the `singletons` library [3], and the *topology* is a value listing, for each vertex, the vertices that may follow it. The action receives the current vertex as a singleton and builds its result with `pureResult`, which requires the constraint `AllowedTransition` for the pair (origin, destination); the identity edge is always allowed. In `crem` 0.1.0.0 this constraint is solved by four instances

marked `INCOHERENT` (we read the source of the package), so resolution is a search for any proof, and a missing edge leaves an unsolved constraint. A transition whose destination is not in the topology of its origin is a compile-time error. The topology is not tied to the input that triggers a transition; this is acknowledged as a limitation by the authors of the library [2]. The idea that types change with protocol state, and that illegal operation sequences can be detected at compile time, is much older [4].

2.2. Imperative encodings and compiler diagnostics

In C, the usual encoding is an `enum` for states, another for events, and nested `switch` statements. A `switch` over an enumeration without a `default` makes the compiler warn about an unhandled enumerator (`-Wswitch`); with a `default` the warning disappears unless a stricter flag (`-Wswitch-enum`) is used. The two enumerations are interchangeable integers in C, so comparing a state with an event compiles. C++11 `enum class` types are distinct and not implicitly convertible [5]. In C++17, `std::variant` and `std::visit` require a callable for every combination of alternatives, and templates with `static_assert` can encode a topology in types [5]. Clang [6] offers warning flags and a static analyser [7]; GHC [8] reports incomplete patterns as a warning (`-Wincomplete-patterns`) that a project may promote to an error.

2.3. Mutation analysis

To learn what a mechanism can detect, we inject defects into a correct program and observe whether the mechanism notices. This is the idea of mutation analysis [9, 10]. Our mutants are deliberate, one per class, and are not a sample of real faults.

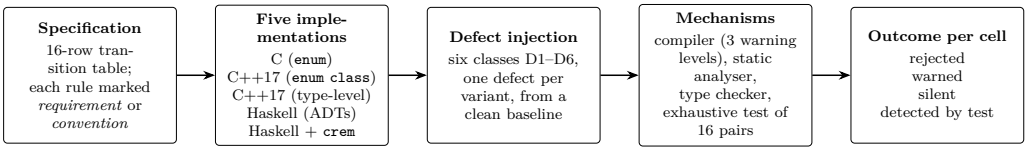


Figure 1: Study design. One transition table is the reference for five implementations, each with a clean baseline and one injected defect per variant; each variant is observed through several mechanisms.

3. Study design

Figure 1 shows the design.

3.1. Subject machine and specification

The machine has four states (`Grounded`, `InFlight`, `EmergencyLanding`, `Shutdown`) and four events (`TakeoffSignal`, `CriticalBattery`, `LandedSignal`, `SystemCutoff`); each event also produces an output message. The specification is a table of 16 rows, one per (state, event) pair, written from the requirement text. Only four rows are stated there (Table 1); the other twelve pairs are a project convention (the state does not change), and the output names are also a convention. The table records the origin of each row so that a reviewer can see what is assumed. The one prohibition in the requirements is the direct edge from `InFlight` to `Shutdown`. All implementations are compared with this table.

3.2. Implementations

Five implementations realise the table (Table 2). Each prints its 16 triples (state, event, destination) so that it can be compared with the table.

Table 1: Destination of each (state, event) pair. R: stated in the requirement text. Unmarked cells keep the state (project convention).

State	`TakeoffSignal`	`CriticalBattery`	`LandedSignal`	`SystemCutoff`
`Grounded`	`InFlight` (R)	–	–	`Shutdown` (R)
`InFlight`	–	`EmergencyLanding` (R)	–	–
`EmergencyLanding`	–	–	`Grounded` (R)	–
`Shutdown`	–	–	–	–

The C++17 typed implementation is our own construction: a primary template `Allowed<From, To>` that is true only for identity, specialisations for the four edges, and a function `go<To>(from)` whose `static_assert` fails when the pair is not allowed. It needs about ten lines beyond the transition function. It is not a recommendation of style; it tests the boundary of the claim that only Haskell can do this.

3.3. Defect classes

Table 3 lists the six defect classes. Each variant contains exactly one defect.

3.4. Mechanisms

- **Compiler, three levels (C-family).** A: `-Wall -Wextra`. B: A plus `-Wswitch-enum -Wenum-compare -Wassign-enum`. W: B plus `-Werror`, the analogue of promoting incomplete patterns to errors in Haskell.
- **Static analyser.** `clang -analyze`.
- **Type checker.** The Haskell and C++17 typed outcomes, including the type-level topology.

Table 2: The five implementations and the toolchain used.

Stack	Encoding	Toolchain
C	`enum` and nested `switch`	clang 14.0.6, default C standard
C++	`enum class` and nested `switch`	clang++ 14.0.6, `-std=c++17`
C++17 typed	`std::variant` states and events; a trait `Allowed` and `static_assert` encode the topology; `std::visit` dispatches	clang++ 14.0.6, `-std=c++17`
Haskell	algebraic data types, one equation per pair, no wildcard	GHC 9.6.7
Haskell + `crem`	singleton vertices, type-level topology, `pureResult`	GHC 9.6.7, `crem` 0.1.0.0, with `-Werror=incomplete-patterns`

- **Exhaustive test.** A harness that prints the 16 triples and compares them with the table. For `crem`, the equivalent is a suite that compares the action's destination and output for every pair with the table.

For Haskell the compiler levels are `-Wall` alone and `-Wall` together with `-Werror=incomplete-patterns`. For `crem` we also injected nine defects into the project code that uses the library (Sect. 4.3), and compiled two further variants of its action for D5 and D6.

3.5. Procedure and reproducibility

Every outcome in this paper comes from scripts that generate the variants by editing the clean baselines, compile them, run the harness and write the

Table 3: Defect classes. The example is the concrete defect injected in the C-family stacks; the same class is instantiated analogously in the others.

Id	Class	Example
D1	omitted pair, no catch-all	the (`InFlight`, `LandedSignal`) case removed
D2	omitted pair with a catch-all	the same case removed and a `default` (or a final wildcard clause) added
D3	forbidden transition	(`InFlight`, `SystemCutoff`) goes to `Shutdown`
D4	wrong destination on an allowed edge	(`Grounded`, `TakeoffSignal`) goes to `Shutdown`, an edge that exists from `Grounded`
D5	state and event confused	a state is compared with an event constant, or an event is used as a destination
D6	value outside the enumeration	the integer 42 returned as a state

results. The baselines print the table exactly (no warning in any mechanism). Versions: clang 14.0.6 (the one shipped with GHC), GHC 9.6.7, `crem` 0.1.0.0, Windows 11. The scripts, tables and logs are part of the artefact (see the data availability statement).

4. Results

4.1. Rejecting forbidden transitions: crem and C++17 give the same matrix

For each of the 16 ordered pairs of vertices we generate a minimal program that requests the transition and compile it. The expected outcome comes from the table: the four edges and the four identities compile, the

Table 4: Compilation of a request for each (origin, destination) pair; identical in `crem` and in C++17 with a type-level topology. ✓: compiles (identity on the diagonal); ×: rejected with the expected cause.

origin \ destination	`Grounded`	`InFlight`	`EmergencyLanding`	`Shutdown`
`Grounded`	✓	✓	×	✓
`InFlight`	×	✓	✓	×
`EmergencyLanding`	✓	×	✓	×
`Shutdown`	×	×	×	✓

other eight pairs must be rejected. A pair counts as rejected only if the compiler reports exactly one error and the error names the pair (for `crem`, `Could not deduce AllowedTransition` for the pair; for C++17, a failed `static_assert` naming `Allowed<From, To>`). All 16 results agree with the table in both stacks, and the two matrices are identical (Table 4).

The answer to RQ2 is therefore no: a forbidden transition can be rejected at compile time without Haskell, with a small amount of template machinery. The matrix checks the *topology*; it is not the same as the state-by-event matrix, which also has 16 cells but concerns the coverage of the transition function and is the subject of D1 and D2.

4.2. Defect classes and mechanisms

Table 5 summarises what each mechanism does with each defect class in each stack.

D1 and D2: omitted cases. In C and C++, an omitted case in a `switch` without `default` is a warning already at level A (the warning is part of `-Wall`); with a `default`, the warning appears only at level B; and with

Table 5: Outcome per defect class and stack. "Error" = the compiler or type checker rejects; "warning" = compiles with a warning; "silent" = compiles without a diagnostic; "test" = the exhaustive test result. [a] the omitted pair's fallback equals the specified behaviour (identity), so the test passes. [b] variant H9, which omits a pair whose specified behaviour is not the identity; the compiler is equally silent and the test fails.

Class	C	C++	C++17 typed	Haskell	Haskell + `crem`
D1	warning (A, B); error with W. Test[a] passes	warning (A, B); error with W. Test[a] passes	error (no matching function)	warning; error with `-Werror=incomplete-patterns`; test: run-time exception	error (exhaustiveness as error)
D2	silent in A; warning in B; error with W. Test[a] passes	silent in A; warning in B; error with W. Test[a] passes	silent. Test[a] passes	silent. Test[a] passes	silent[b]; test fails[b]
D3	silent; test detects	silent; test detects	error (`static_assert`)	silent; test detects	error (no proof of the edge)
D4	silent; test detects	silent; test detects	silent; test detects	silent; test detects	silent; test detects
D5	silent; test detects only by numeric coincidence	error (distinct types)	error	error (type mismatch)	error (type mismatch)
D6	silent; test detects	silent (`static_cast`); test detects	not applicable (closed sum)	error (no `Num` instance)	error (no `Num` instance)

`-Werror` (level W) both become errors. In Haskell the incomplete-patterns warning is part of `-Wall` and becomes an error with a flag that the `crem` project sets. In C++17 with `std::visit`, an omitted pair is an error because the visitor has no overload for that combination. A catch-all clause hides the omission from the default diagnostics of every stack. Only C and C++ have

a stricter flag, `-Wswitch-enum`, that exposes it (levels B and W), at the price of also flagging legitimate `default` clauses; in C++17 typed, Haskell and `crem` nothing in our configurations flags it. Outside `std::visit`, where the omission is a type error by construction, the difference between "warns" and "rejects" is a matter of configuration, not of language.

In the `crem` variant of D2 (H9), the actions for `Grounded` with `TakeoffSignal` were removed and a final clause `avionicsAction s _ = pureResult MsgIgnored s` (an identity, which is always an allowed edge) was added: the library compiles without a diagnostic, and the behaviour changes. In the variants of the other stacks, the omitted pair happened to be one whose specified behaviour is the identity, so the test passes while the compiler's silence or warning is the only signal. We return to this asymmetry in Sect. 4.5.

D3: forbidden transition. It is invisible to the C and C++ compilers and analyser and to plain Haskell, and is detected by the exhaustive test. It is rejected at compile time by the C++17 typed construction and by `crem`. This is the class where the type-level topology adds something, and it adds the same in the two stacks that have it.

D4: wrong destination on an allowed edge. Every stack is silent at compile time, and only the test detects it. The reason is structural: the topology fixes which vertices may follow which, not which event leads to which vertex. In the C++17 typed variant, redirecting the take-off from `InFlight` to `Shutdown` still satisfies `Allowed<Grounded, Shutdown>`. No compiler or analyser in the study detected this class.

D5 and D6. Distinct types for states and events turn the confusion of D5 into an error in every stack except C, where the two enumerations convert to each other; the test detects it only because the numeric values of the two constants happen to differ for the compared pair. An out-of-range value (D6) is an error in Haskell, impossible by construction in the C++17 typed variant (a variant of unrelated types has no such value), and silent in C and C++ (a `static_cast` suffices). In `crem` both classes were injected into the action itself (an event returned as the destination, and the integer 42 returned as the destination); both are type errors, the second reported both as a missing `Num` instance for the vertex singleton and as an unprovable edge.

Static analyser. `clang -analyze` reported nothing for any variant that compiles. This holds for this analyser and version with its default checks, and says nothing about static analysis in general.

4.3. The crem-based project code under mutation

To see whether the pattern holds in code written against the library, and not only in a minimal example, we injected nine defects, one at a time, into the project's `crem` implementation of the machine and of the components around it (the battery machine and an environment machine of a larger system), and ran five test suites on each. The `crem` library itself was not modified. Two defects were rejected by the compiler: an illegal edge in the action without a change to the topology, and an omitted pair. Seven were accepted and killed by at least one suite: an illegal edge added to both topology and action, a wrong destination on an allowed edge, swapped events, a wrong output, two defects in the components around the machine, and the

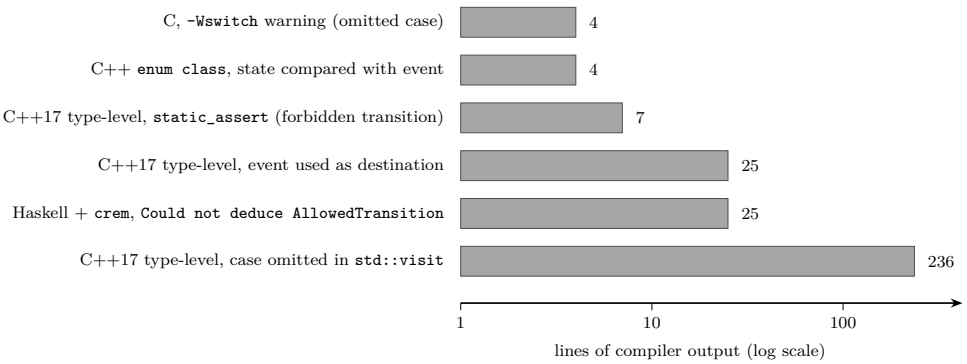


Figure 2: Size of the compiler output for six defects (logarithmic scale).

catch-all variant above. No mutant survived. The full list of mutants is in the artefact. Compile-time rejection covered two of nine defects; the remaining seven were found by tests, among them those of D4 and of the output of an action, which the type does not mention.

4.4. Size of the diagnostics

Figure 2 shows the size, in lines, of the compiler output for six rejected or warned defects. The `-Wswitch` warning of C and the type mismatch of C++ `enum class` have 4 lines; the `static_assert` that rejects a forbidden transition in C++17 has 7; the `crem` error for the same defect has 25 lines (1 636 bytes); and an omitted case in `std::visit` has 236 lines (35 863 bytes) of template instantiation messages. The scripts measure these sizes on every run (the byte counts include file paths and change with the location of the checkout; the line counts do not). Size is not readability: there was no reader study. It is an upper bound on what the engineer must scan to find the cause.

4.5. What the exhaustive test adds, and what it cannot see

Over a finite domain of 16 pairs, the exhaustive test detected every defect that changes the behaviour in every stack in which it compiles (D3, D4, D5 and D6 in C; D3, D4 and D6 in C++; D4 in C++17 typed; D1, D3 and D4 in plain Haskell, where D1 appears as a run-time exception; and all `crem` mutants that compile). It cannot see a latent defect whose consequence equals the specified behaviour: an omitted pair whose fallback is the identity passes the test (D1 and D2 in C and C++; D2 in C++17 typed and in Haskell). That is exactly the class that compilers flag when warnings are errors. The two mechanisms are complementary: the test catches what changes behaviour, the strict compiler what *would* change it if the specification changed. Note that the test must include the *events*: a test of the topology alone, such as the matrix of Sect. 4.1, cannot detect D4.

5. Discussion

What compile-time checking buys. In this study, compile-time checking advanced the detection of two classes: omitted cases (when warnings are errors, or by construction with `std::visit`) and forbidden transitions (when the topology is in the types). For those classes it also removes the need to write a test: the edge-specific tests for D3 are unnecessary where the type-level topology exists. This is a real gain in the position and cost of detection. It is not a gain in coverage: the exhaustive test detected the same behavioural defects.

It is not specific to Haskell. A construction in C++17 of about ten lines reproduces the 16-pair rejection matrix of `crem`. The cost is in the diagnostics

(up to 236 lines for an omitted pair) and in the constructs the programmer must know. The claim that Haskell detects defects that C or C++ cannot would be too broad; what is verifiable is that certain topology constraints are checked at compile time in `crem`, which anticipates their detection and dispenses with specific tests for those edges.

The event–destination gap is shared. The one class that no compiler or analyser detected, D4, is the one that matters for behaviour: the wrong consequence of an event on an allowed edge. Tying events to edges in the types is possible in principle with indexed monads [11] or dependently typed languages [12]; none of the stacks here does it, and we did not evaluate these alternatives. With the tools as they are, the defence is a specification that includes events, checked by an exhaustive test.

Practical recommendations, within this study. (i) Compile with warnings as errors, and in particular promote exhaustiveness to an error. (ii) Avoid catch-all clauses in transition functions, or review them as part of the specification. (iii) Test the events, not only the edges. (iv) If a type-level topology is wanted, a library is not the only way.

5.1. Threats to validity

One machine. The subject has four states and four events. Larger machines and other topologies may change the cost, but the structure of the classes does not depend on size; the cost of diagnostics and of compilation does, and we did not measure compilation time.

Authorship and review. The same person wrote the five implementations, the table, the defects and the scripts. The C and C++ code, in particular,

was written by an author with little experience in those languages, and has not been reviewed by an independent C or C++ expert; the typed C++17 construction is one possible design among others (for instance `constexpr` tables or C++20 concepts). A review is pending. The defects are deliberate and are not a sample of real faults.

One compiler per language. We used clang 14.0.6 for C and C++ and GHC 9.6.7 for Haskell, on one operating system. GCC, MSVC and other analysers may behave differently; `clang-tidy` was not available in the toolchain used.

Proxies. The size of the diagnostics is a proxy for readability; maintenance effort was not measured.

Procedure. Every figure of the paper comes from a recorded run, and the scripts check that the clean baselines print the table exactly, so that the variants differ from a verified baseline by one defect.

6. Related work

This section is not a systematic review; a protocol for one exists in the artefact but the search has not been carried out, and the paper does not claim novelty beyond what is stated in Sect. 1.2.

Typed encodings of state machines in Haskell are the subject of `crem` [2], which builds on singletons [3], and of indexed monads [11]. Typestate [4] is the classic formulation of protocol-dependent types. The comparison of finite-state machines in imperative and in pure functional languages was also treated in Russian-language work [13], which we cite only for the general observation that imperative encodings use enumerations and a mutable

state variable. Mutation analysis [9, 10] is the basis of our defect injection. What we add is a controlled comparison of concrete mechanisms across five implementations of the same machine, including a C++17 construction that matches a type-level library on the topology matrix.

7. Conclusions

For a four-state machine, a type-level topology rejects exactly the eight vertex pairs outside the topology, both in `crem` and in a C++17 construction of about ten lines. Rejecting a forbidden transition at compile time is not specific to Haskell. Omitted cases are stopped by default only where the encoding makes them a type error (`std::visit`); elsewhere, only when warnings are errors, and a catch-all clause hides them from the default diagnostics of every stack. The class that matters for behaviour, a wrong destination on an allowed edge, was detected by no compiler or analyser and by every exhaustive test. Compile-time checking anticipates detection and removes edge-specific tests; it does not make behavioural correctness equivalent to type safety. Future work includes other machines and scales, an independent review of the C and C++ code, other compilers and analysers, and a study of maintenance effort.

CRediT authorship contribution statement

[To be completed by the authors, using the CRediT roles required by the journal (e.g. Conceptualization, Methodology, Software, Investigation, Writing – original draft, Supervision).]

Declaration of competing interest

[To be completed by the authors.]

Funding

[To be completed by the authors: list the funding sources, or, if there were none, state that this research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.]

Data availability

The scripts, the transition table, the C, C++ and Haskell sources, the generated variants and all logs are in the artefact of this work: a frozen snapshot of the `avionics-crem` repository (commit `9689ce8`) and the added folders with the experiment scripts. **[Public repository URL and archived DOI to be added before submission; the repository was private at the time of writing.]** One script rebuilds and reruns everything.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

[To be defined with the external advisor.]

References

[1] G. Hutton, Programming in Haskell, 2nd Edition, Cambridge University Press, Cambridge, UK, 2016.

[2] M. Perone, G. Karachalias, Crème de la crem: Composable representable executable machines (architectural pearl), preprint, arXiv:2307.09090. Presented at the FUNARCH workshop, ICFP 2023. Library: crem 0.1.0.0, `https://hackage.haskell.org/package/crem-0.1.0.0` (2023).

[3] R. A. Eisenberg, S. Weirich, Dependently typed programming with singletons, in: Proceedings of the 2012 Haskell Symposium (Haskell '12), ACM, New York, NY, USA, 2012, pp. 117–130.

[4] R. E. Strom, S. Yemini, Typestate: A programming language concept for enhancing software reliability, IEEE Trans. Softw. Eng. SE-12 (1) (1986) 157–171. doi:10.1109/TSE.1986.6312929.

[5] ISO/IEC, ISO/IEC 14882:2017, Programming languages – C++, 5th Edition, International Organization for Standardization, Geneva, Switzerland, 2017.

[6] The LLVM Project, Clang: a C language family frontend for LLVM, version 14.0.6, `https://clang.llvm.org/`, version 14.0.6, as shipped with GHC 9.6.7 for Windows.

[7] The LLVM Project, Clang static analyzer, `https://clang-analyzer.llvm.org/`, used through `clang --analyze`, version 14.0.6.

[8] The GHC Team, The Glasgow Haskell Compiler, version 9.6.7, `https://www.haskell.org/ghc/`.

[9] R. A. DeMillo, R. J. Lipton, F. G. Sayward, Hints on test data selection: help for the practicing programmer, Computer 11 (4) (1978) 34–41. doi:10.1109/C-M.1978.218136.

[10] Y. Jia, M. Harman, An analysis and survey of the development of mutation testing, IEEE Trans. Softw. Eng. 37 (5) (2011) 649–678. doi:10.1109/TSE.2010.62.

[11] R. Atkey, Parameterised notions of computation, J. Funct. Program. 19 (3–4) (2009) 335–376. doi:10.1017/S095679680900728X.

[12] E. Brady, Idris, a general-purpose dependently typed programming language: design and implementation, J. Funct. Program. 23 (5) (2013) 552–593. doi:10.1017/S095679681300018X.

[13] Y. M. Malakhovski, A. A. Shalyto, Konechnye avtomaty v chistykh funktsionalnykh yazykakh programmirovaniya [finite state machines in pure functional programming languages] (in russian), RSDN Magazine (1) (2009).